SDS - REQ-DA-003 나머지 코드값 enum 전환과 범위 검증

작성일 2026-10-17 · 근거: 요구사항정의서_엑셀양식_v3_1_3.xlsx (F열) · SRS 없음 · 대상: D:₩legacy-portal

1. 개요

항목	내용
요구사항 ID	REQ-DA-003
요구사항명	나머지 코드값 enum 전환과 범위 검증
대 / 중 / 소분류	PORTAL / Data / Enum
우선순위	Medium
진행현황	설계중
수용여부 · 요청자	수용 · 포털팀 (미정)
엑셀 비고	리팩토링 백로그 R07(보류). REPORT.md 9장 남은 과제.
요약	결재 유형 · 우선순위, 공지 상태 · 분류, 일정 상태, 사용자 직급의 int 코드를 결재 상태(ApprovalStatus)와 같은 방식의 enum 으로 바꾸고 범위 밖 값을 거절한다. 현재는 미착수(리팩토링 백로그 R07 보류)로 모든 필드가 int 이며 type 5 같은 값도 그대로 저장된다. 설계상 핵심 제약은 특성화 테스트가 Notice.setStatus(2) 로 '알수없음' 라벨을 확인한다는 점으로, 공지 상태는 enum 필드 + AttributeConverter 로 바꾸면 이 테스트가 깨진다. 따라서 공지 상태만 int 저장 + enum 해석으로 설계한다.

2. 근거

SRS 없음 (엑셀 F열 · 소스만으로 분석). docs₩SRS 폴더가 존재하지 않아 아래 F열 상세내용 원문과 D:₩legacy-portal 소스(src₩main · src₩test) · CLAUDE.md 불변 규칙만을 근거로 삼는다.

```
[개요]
결재 유형 · 우선순위, 공지 상태 · 분류, 일정 상태, 사용자 직급의 숫자 코드를 enum 으로 바꾸고, 범위 밖 값을 받지 않게 한다.

[상세 기능 요구사항]
1) 각 코드에 대해 결재 상태와 같은 방식(code · label · fromCode · AttributeConverter)의 enum 을 만든다.
2) 엔티티 필드를 enum 으로 바꾸되 DB 저장값은 REQ-DA-001 의 정수 그대로 둔다.
3) API 로 범위 밖 코드(예: type 5, priority 7)가 들어오면 거절한다.
4) 공지 상태 표시명(임시 · 게시 · 내림)을 enum label 로 옮긴다.

[입력 / 출력]
- 입력: 현재 정수 코드 필드
- 출력: enum 필드, Converter, 범위 밖 값 거절

[제약 조건 / 비기능 요구사항]
- DB 저장값 · JSON 정수 코드 형식을 바꾸지 않는다.
- 범위 밖 값 거절은 동작 변경이므로 REQ-NF-003 입력 검증과 함께 적용한다.

[검증 기준]
- 전환 후에도 JDBC 로 읽은 컬럼 값이 전환 전과 같다.
- type=5 로 기안하면 거절되고 저장되지 않는다.
```

[작업 대상]
legacy-portal (D:₩legacy-portal) Spring Boot 3.2 / Java 21
src₩main₩java₩kr₩hull₩portal₩approval₩Approval.java
src₩main₩java₩kr₩hull₩portal₩notice₩Notice.java
src₩main₩java₩kr₩hull₩portal₩schedule₩Schedule.java
src₩main₩java₩kr₩hull₩portal₩user₩User.java

[Worker Settings]
- MainWorker : claude-main
- SubWorker : codex-critic
- MainWorker-SubAgent : database-expert

3. 분석 결과

3.1 추적표 (F열 문장 ↔ 소스)

항목	요구사항(F열 문장)	SRS 근거	소스 위치	판정	비고
기능 1	각 코드에 대해 결재 상태와 같은 방식(code · label · fromCode · AttributeConverter)의 enum 을 만든다.	SRS 없음	패턴만 존재: ApprovalStatus.java:13-52 · ApprovalStatusConverter.java:13-25. ApprovalType · ApprovalPriority · NoticeStatus · NoticeCategory · ScheduleStatus · UserRole 클래스 없음(src₩main 파일 목록)	**미충족**	Approval.java:23 · Notice.java:17 · Schedule.java:17 · User.java:12 주석에 'R07 단계로 남김' 명시
기능 2	엔티티 필드를 enum 으로 바꾸되 DB 저장값은 REQ-DA-001 의 정수 그대로 둔다.	SRS 없음	Approval.java:48 int type · :54 int priority, Notice.java:34-35 int status · category, Schedule.java:32 int status, User.java:27 int role	**미충족**	전부 int. 저장값 자체는 REQ-DA-001 대로 정수
기능 3	API 로 범위 밖 코드(예: type 5, priority 7)가 들어오면 거절한다.	SRS 없음	CreateApprovalRequest.java:18-32 검증 없음(Integer type · priority 를 그대로 toCommand), Approval.java:105-106 command.type() · priority() 그대로 대입	**미충족**	type 5 · priority 7 이 저장되고 200 응답(R11 이전 컨트롤러 주석 '잘못된 값(5 등)도 그냥 통과' - git 17760e4^)
기능 4	공지 상태 표시명(임시 · 게시 · 내림)을 enum label 로 옮긴다.	SRS 없음	Notice.java:105-110 statusLabel() 의 if 사다리(0 임시 · 1 게시 · 9 내림 · 그 밖 알수없음), NoticeService.java:106-108 위임	**미충족**	엔티티 메서드까지는 옮겨졌으나 enum 은 아님
입력	입력: 현재 정수 코드 필드	SRS 없음	Approval.java:48 · 54, Notice.java:34-35, Schedule.java:32, User.java:27	**충족**	전환 대상 6개 필드 현황 확인됨
출력	출력: enum 필드, Converter, 범위 밖 값 거절	SRS 없음	해당 클래스 · 검증 없음	**미충족**	
제약 1	DB 저장값 · JSON 정수 코드 형식을 바꾸지 않는다.	SRS 없음	전환 전이라 현행 유지: ApprovalResponse.java:20-22 int type · status · priority, User 엔티티 직접 직렬화 int role	**충족**	전환 미착수 상태의 충족. 전환 후에도 유지돼야 함(9절 안전망)

항목	요구사항(F열 문장)	SRS 근거	소스 위치	판정	비고
제약 2	범위 밖 값 거절은 동작 변경이므로 REQ-NF-003 입력 검증과 함께 적용한다.	SRS 없음	REQ-NF-003 진행현황 '기획중', 소스에 입력 검증(jakarta.validation · @Valid) 없음	**확인 불가**	적용 순서 · 묶음은 아직 실행되지 않은 계획 - 소스로 판정 불가

3.2 판정 요약

충족 2 / 부분 충족 0 / 미충족 5 / 불일치 0 / 확인 불가 1 (총 8건)

3.3 검증 기준 ↔ 테스트

검증 기준(F열)	고정하는 테스트	비고
전환 후에도 JDBC 로 읽은 컬럼 값이 전환 전과 같다.	결재 status 만 있음: ApprovalStatusPersistenceTest#임시저장_결재는_DB에_정수_0으로_저장된다 · #승인된_결재는_DB에_정수_2로_저장된다 · #취소된_결재는_DB에_순번_4가_아니라_정수_9로_저장된다. type · priority · 공지 · 일정 · role 컬럼은 테스트 없음	전환 전에 먼저 추가해야 하는 안전망(9절)
type=5 로 기안하면 거절되고 저장되지 않는다.	테스트 없음	현행은 저장됨. 동작 변경 테스트이므로 REQ-NF-003 과 같은 커밋에서 새 테스트로 추가

4. 구성 요소 설계

enum (신규, 소속 도메인 이름)	code · label	Converter	적용 필드
ApprovalType	EXPENSE 1 지출 · VACATION 2 휴가 · PURCHASE 3 구매 · ETC 4 기타	ApprovalTypeConverter	Approval.type
ApprovalPriority	LOW 1 낮음 · NORMAL 2 보통 · HIGH 3 높음	ApprovalPriorityConverter	Approval.priority
NoticeStatus	DRAFT 0 임시 · PUBLISHED 1 게시 · TAKEN_DOWN 9 내림	없음 (int 저장 유지 - 아래 설계 판단)	Notice.status (해석용)
NoticeCategory	GENERAL 1 일반 · EMERGENCY 2 긴급 · PERSONNEL 3 인사	NoticeCategoryConverter	Notice.category
ScheduleStatus	PLANNED 0 예정 · CONFIRMED 1 확정 · CANCELED 9 취소	ScheduleStatusConverter	Schedule.status
UserRole	STAFF 1 사원 · TEAM_LEADER 2 팀장 · EXECUTIVE 3 임원	UserRoleConverter	User.role

공통 시그니처 (ApprovalStatus 와 같은 모양)

```
public enum ApprovalType {
    EXPENSE(1, "지출"), VACATION(2, "휴가"), PURCHASE(3, "구매"), ETC(4, "기타");
    public int getCode()
    public String getLabel()
    public static ApprovalType fromCode(int code)          // 정의 밖 code → IllegalArgumentException
    public static boolean isDefined(int code)               // DTO 검증용 (REQ-NF-003)
}
```

```
@Converter
public class ApprovalTypeConverter implements AttributeConverter<ApprovalType, Integer> {
    public Integer convertToDatabaseColumn(ApprovalType type)    // null → null, 그 밖 → getCode()
    public ApprovalType convertToEntityAttribute(Integer code)   // null → null, 그 밖 → fromCode
}

// 엔티티 - public 계약(int getter · setter) 현행 유지, enum 접근은 get/set 접두 없는 메서드로 (JSON 필드 증가 방지)
public int  Approval.getType()              // type == null ? 0 : type.getCode()  (new Approval() 기본값 0 보존)
public void Approval.setType(int type)      // ApprovalType.fromCode(type)
public ApprovalType Approval.approvalType()
public int  User.getRole()  /  public void User.setRole(int role)  /  public UserRole userRole()
public boolean User.hasApprovalAuthority()  // userRole().getCode() >= UserRole.TEAM_LEADER.getCode() - 판정 결과 동일

// 공지 상태 - int 필드 유지 + enum 해석
private int status;                                        // DB 0/1/9 그대로, setStatus(int) 는 어떤 값도 받음
public static Optional<NoticeStatus> NoticeStatus.fromCode(int code)
public String Notice.statusLabel()                          //
NoticeStatus.fromCode(status).map(NoticeStatus::getLabel).orElse("알수없음")
```

설계 판단 - 공지 상태를 Converter 대상에서 제외

특성화 테스트 NoticeServiceCharacterizationTest#상태_라벨은_0임시_1게시_9내림_그밖은_알수없음이다(NoticeServiceCharacterizationTest.java:267-278)는 notice.setStatus(2) 후 statusLabel 이 "알수없음" 인지 확인한다. 공지 상태를 enum 필드로 바꾸면 2 를 담을 수 없어 setStatus(2) 가 예외를 내고, 테스트 본문은 수정 금지이므로 이 경로는 쓸 수 없다. 따라서 공지 상태는 ApprovalAction 처럼 Optional 을 돌려주는 fromCode 로 해석만 하고, 필드 · DB 는 int 그대로 둔다. 라벨은 enum 이 갖게 되어 기능 4 는 충족된다.

나머지 필드는 테스트에서 정의된 값만 쓰는 것을 확인했다(src₩test 의 new User(...) role 1 · 2 · 3, create(...) type 1 · 2 · 3 · 4 범위 · priority 1 · 2, Schedule setStatus 1 · 9, 공지 category 1 · 2 · 3 상수). 단 new Approval() 의 int 기본값 0(type · priority)은 정의 밖이므로 getter 가 null 을 0 으로 돌려주게 해 보존한다.

5. 인터페이스 설계

URL · 필드 이름 · JSON 정수 형식은 그대로다. 달라지는 것은 범위 밖 값의 처리뿐이며 이것은 REQ-NF-003 의 검증 응답 형식(REQ-NF-002 오류 형식)을 따른다.

API	필드	허용 값	범위 밖일 때(전환 후)	현행
POST /api/approvals	type	1 · 2 · 3 · 4	400 + 필드 사유 'type', 저장 안 함	그대로 저장, 200
POST /api/approvals	priority	1 · 2 · 3	400 + 'priority'	그대로 저장, 200
POST /api/notices (REQ-IF-003)	category	1 · 2 · 3	400 + 'category'	API 없음
모든 응답	type · priority · status · category · role	정수	정수 그대로	정수

```
POST /api/approvals
{"title":"노트북","content":"","type":5,"priority":2,"drafterId":1,"approverId":2}
→ (전환 후) 400  {"code":"INVALID_INPUT","message":"...","fields":{"type":"정의되지 않은 결재 유형: 5"}}
// 형식은 REQ-NF-002 확정안을 따름
→ (현행)    200  {"id":..,"type":5,...}
```

6. 데이터 설계

테이블.컬럼	현재 매핑	전환 후 매핑	DB 저장값(불변)
approval.type	int	@Convert(ApprovalTypeConverter)	1 · 2 · 3 · 4
approval.priority	int	@Convert(ApprovalPriorityConverter)	1 · 2 · 3
notice.status	int	int (변경 없음, NoticeStatus 로 해석)	0 · 1 · 9
notice.category	int	@Convert(NoticeCategoryConverter)	1 · 2 · 3
schedule.status	int	@Convert(ScheduleStatusConverter)	0 · 1 · 9
users.role	int	@Convert(UserRoleConverter)	1 · 2 · 3

@Enumerated(STRING) · @Enumerated(ORDINAL) 은 쓰지 않는다(CLAUDE.md 불변 규칙). ORDINAL 이면 9 가 2 로, 우선순위 1 이 0 으로 어긋난다. int 기본형 컬럼은 NOT NULL 이었으므로 Converter 필드에도 @Column(nullable = false) 를 붙여 스키마를 같게 한다(Approval.java:52 와 같은 방식).

상태 전이(바뀌지 않음, 상수만 enum 으로)

대상	전이	근거(현행)
공지	0 임시 → 1 게시 (팀장 이상) / 1 · 9 는 변화 없음	Notice.java:84-93
일정	0 예정 → 1 확정 (소유자) / 1 · 9 는 변화 없음, 9 는 겹침 판정 제외	Schedule.java:95-117

7. 처리 흐름

단계	처리	비고
0 안전망 추가	type · priority · notice.status · category · schedule.status · role 의 JDBC 직접 조회 테스트 작성 · green 확인	전환 전에
1 enum 추가	6개 enum (+ Converter 5개)	동작 무변경
2 필드 전환	엔티티 필드를 enum 으로, public int getter · setter 는 code 로 위임	한 필드 한 커밋, 매번 ./mvnw test green
3 내부 사용처 교체	EXPENSE_TYPE · HIGH_PRIORITY · TEAM_LEADER_ROLE · DRAFT_STATUS 등 상수 → enum 비교	Approval.java:36-37, Notice.java:23-26, Schedule.java:23-25, User.java:30
4 라벨 이동	Notice.statusLabel → NoticeStatus.getLabel (+ 알수없음 fallback)	기능 4
5 범위 밖 거절	CreateApprovalRequest 등 DTO 에서 isDefined 검사 → 400 (REQ-NF-003 과 같은 커밋)	동작 변경 - 별도 단계

1~4 는 리팩토링(동작 보존), 5 는 재설계(동작 변경)다. 두 단계를 한 커밋에 섞지 않는다.

8. 예외 · 경계 처리

경계 · 예외	현행 동작	전환 후	레거시 보존 여부
Notice.setStatus(2) → statusLabel	"알수없음"	"알수없음" (int 유지 + Optional 해석)	보존 (특성화 테스트)
new Approval().getType() · getPriority()	0	0 (null → 0)	보존
서비스 create(type 5) 직접 호출	저장됨	IllegalArgumentException (setType · draft 에서 fromCode)	변경 - REQ-NF-003 단계에서만 적용, 그 전에는 int 경로 유지
API type 5 · priority 7	200 · 저장	400 · 저장 안 함	변경 (재설계)
DB 에 정의 밖 값이 이미 있음	그대로 읽힘	Converter 에서 예외 - 엔티티 로드 실패	데이터 이행 전 점검 쿼리 필요(10절)
role 판정 role ≥ 2	팀장 · 임원 true	같음	보존

9. 검증 설계

검증 기준	현재 테스트	추가 제안 테스트
전환 후 JDBC 컬럼 값 동일	결재 status 만 (ApprovalStatusPersistenceTest)	CodeValuePersistenceTest#결재_유형과_우선순위는_DB에_정수_그대로_저장된다, #공지_상태와_분류는_DB에_정수_그대로_저장된다, #일정_상태는_DB에_정수_그대로_저장된다, #사용자_직급은_DB에_정수_그대로_저장된다 (REQ-DA-001 9절과 같은 테스트 - 전환 전에 작성)
type=5 기안 거절 · 미저장	테스트 없음	ApprovalInputValidationTest#유형_5로_기안하면_400이고_저장되지_않는다, #우선순위_7로_기안하면_400이다
(설계) enum code 표	테스트 없음	CodeEnumTest#결재_유형_enum의_code는_1부터_4다, #정의되지_않은_code는_예외가_난다

```
@SpringBootTest
@AutoConfigureMockMvc
@Transactional
class ApprovalInputValidationTest {
    @Autowired
    private MockMvc mockMvc;
    @Autowired
    private ApprovalRepository approvalRepository;

    @Test
    void 유형_5로_기안하면_400이고_저장되지_않는다() throws Exception {
        long countBefore = approvalRepository.count();
        String invalidTypeRequest = """
                {"title":"노트북","content":"","type":5,"priority":2,"drafterId":1,"approverId":2}""";


mockMvc.perform(post("/api/approvals").contentType(MediaType.APPLICATION_JSON).content(invalidTypeRequest
))
                .andExpect(status().isBadRequest());
        assertThat(approvalRepository.count()).isEqualTo(countBefore);
```

```
    }
}
```

기존 특성화 테스트(정수 리터럴 0/1/2/3/9 · 1~4 호출)는 enum 상수 비교로 바꾸지 않고 그대로 green 이어야 한다.

10. 검토 의견 · 후속 과제

보존 대상

- DB 저장 정수값 6종(REQ-DA-001) · 응답 JSON 정수 형식 · 필드 수
- 엔티티 public int getter · setter 시그니처(특성화 테스트가 setStatus · new User(.., int, ..) 로 준비)
- 공지 상태의 '그 밖 = 알수없음' 라벨 동작, new Approval() 기본값 0
- 권한 판정(role ≥ 2) · 공지 게시 권한과 결재 권한의 분리(User.java:13-16 우연 중복 주석)

개선 대상

- [미충족] enum 6종 · Converter 5종 미구현 - 4절 설계안
- [미충족] 범위 밖 코드 거절 - 동작 변경이므로 REQ-NF-003 과 함께, 특성화 테스트 수정이 아닌 새 테스트로
- [미충족] 공지 상태 라벨 enum 이동 - int 저장 유지 + NoticeStatus 해석(4절 설계 판단)
- [확인 불가] REQ-NF-003 과 묶어 적용 - 계획 사항, 착수 시 커밋 단위로 확인
- 운영 DB 이행 전 점검: select type, count(*) from approval where type not in (1,2,3,4) group by type 등 정의 밖 값 존재 여부 확인

검토 의견: F열 기능 1 은 '모든 코드에 AttributeConverter' 를 요구하지만, 공지 상태는 특성화 테스트 불변 규칙과 충돌한다. F열을 '공지 상태는 int 저장 + enum 해석(라벨 · 비교)' 으로 보완하거나, 재설계 단계(REQ-NF-002)에서 해당 특성화 테스트를 새 기대값으로 교체하는 별도 커밋 이후에 Converter 를 적용하는 것 중 하나를 선택해야 한다.

연계 요구사항: REQ-DA-001(저장값 보존 - 안전망), REQ-NF-003(입력 검증 · 함께 적용), REQ-NF-002(오류 응답 형식), REQ-IF-003(공지 API 의 category 입력).